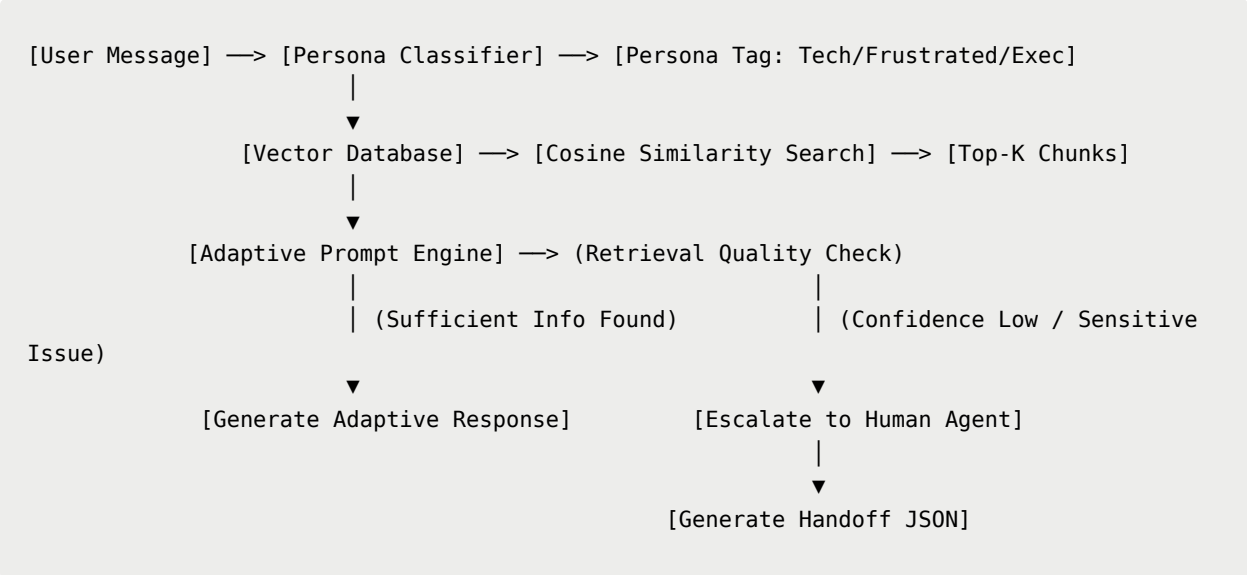
Adsparkx AI Assignment Reference Document

Building a Persona-Adaptive Customer Support Agent: Implementation Guide.

1. Objective

The primary objective of this project is to build an intelligent customer support agent that adapts its response style based on the customer's communication persona.

Core System Workflow



2. Technical Requirements

To ensure a smooth setup, install the following required languages, libraries, and frameworks.

System Requirements

- **Operating System**: Windows 10/11, macOS, or Linux
- **Language**: Python 3.11 or higher
- **API Credentials**: Google Gemini API Key

Python Library Dependencies

Create a virtual environment and install the following core packages:

Library	Version	Purpose
`google-genai`	`>=0.1.0`	Official Google SDK for Gemini LLMs and Embeddings
`streamlit`	`>=1.30.0`	Interactive Chat Web UI
`chromadb` (or `faiss-cpu`)	`>=0.4.0`	Local Vector Database for index retrieval
`langchain`	`>=0.1.0`	Document chunking and orchestrating helper utilities
`pypdf`	`>=3.0.0`	Native PDF reading and parsing capabilities
`python-dotenv`	`>=1.0.0`	Environment variable management for API keys

3. Project Structure

A clean repository structure makes code maintenance easy and modular. Organize your repository as follows:

```
persona-support-agent/
│
├── data/
│   ├── api_troubleshooting.md
│   ├── billing_policy.txt
│   └── password_reset_guide.pdf     <-- (Required PDF Document)
│
├── src/
│   ├── __init__.py
│   ├── config.py                     <-- App configuration and thresholds
│   ├── classifier.py                 <-- Persona detection logic
│   ├── rag_pipeline.py               <-- Chunker, Vector DB creator, and Retriever
│   ├── generator.py                  <-- Persona-based prompt compiler and LLM Caller
│   └── escalator.py                  <-- Confidence thresholds & escalation handoff gene
rator
│
```

```
├── app.py                          <-- Main Streamlit / Gradio Web UI
├── requirements.txt                <-- List of pip packages
├── .env                            <-- Local secret variables (Git-ignored)
└── README.md                       <-- Setup instructions & architectural diagram
```

4. Step-by-Step Implementation Guide

This project is broken down into five core architectural milestones.

Step 1: Environment and Knowledge Base Preparation

First, set up your development workspace and gather your document files.

1. **Configure** `.env`: Store your Gemini API key inside a `.env` file at the root level:

   ```
   GEMINI_API_KEY="your_actual_gemini_api_key_here"
   ```

2. **Generate Knowledge Base Documents**: Write or obtain 10–20 realistic help desk articles. Populate the `data/` directory with plain text files (`.txt`), markdown explanations (`.md`), and at least one instructional PDF (`.pdf`) covering password recoveries, API authentications, and payment pathways.

Step 2: Customer Persona Classification

When a message arrives, we pass it to a lightweight prompt designed to run classification.

- **How it works**: The user's query is analyzed by Gemini using a structured output layout. We instruct the model to return a strict JSON payload indicating the matched persona alongside a justification.
- **Mathematical Context**: To understand how users form patterns, you can conceptually visualize their text as a high-dimensional vector. The classification stage maps the conversational semantic footprint of the text into discrete classes:$$\text{Persona} \in \{\text{"Technical Expert"}, \text{"Frustrated User"}, \text{"Business Executive"}\}$$

Step 3: Designing and Building the RAG Pipeline

To prevent hallucinations, the model must *only* answer using factual documents from our `/data` folder. A Retrieval-Augmented Generation (RAG) system solves this by retrieving relevant document snippets and injecting them into the LLM's prompt context. Here is a granular, step-by-step breakdown of how to build this pipeline:

1. Document Ingestion & Parsing

Your raw customer support data resides in files with different formats (such as `.txt`, `.md`, and `.pdf`). Before processable text chunks can be constructed, these files must be parsed:

- **Plain Text and Markdown (`.txt`, `.md`)**: These files can be opened and read natively using standard Python file operations:

```
with open("data/api_troubleshooting.md", "r", encoding="utf-8") as f:
    text = f.read()
```

- **PDF Documents (`.pdf`)**: PDFs store text using complex geometric layouts. You must use a library like `pypdf` to extract text page-by-page:

```
from pypdf import PdfReader
reader = PdfReader("data/password_reset_guide.pdf")
pdf_text = ""
for page in reader.pages:
    pdf_text += page.extract_text() + "\n"
```

2. Strategic Document Chunking

LLMs have context window constraints and processing large documents as a single block is inefficient. Therefore, parsed text must be broken down into smaller, readable pieces called **chunks**:

- **Chunk Size**: This is the character length of each individual block (e.g., 500 characters). It should be small enough to stay focused on a single topic, but large enough to contain complete, meaningful thoughts.
- **Chunk Overlap**: This refers to the number of characters shared between adjacent chunks (e.g., 50 characters). Maintaining an overlap prevents

critical context (such as an API endpoint or password reset step) from being sliced in half across chunk boundaries.

- **Splitting Algorithm**: We use a `RecursiveCharacterTextSplitter`. This algorithm attempts to split text using natural separators in order of importance—paragraphs (`\n\n`), sentences (`\n`), words (` `), and finally individual characters (`""`)—guaranteeing that paragraphs and sentences remain intact where possible.

3. Vector Embedding Generation

Computers do not natively understand semantic language. To represent meaning mathematically, we convert each text chunk into a **dense vector embedding** using Gemini's state-of-the-art `text-embedding-004` model.

- An embedding is a list of floating-point numbers (usually 768 dimensions) representing the semantic footprint of a text block.
- Two sentences with similar meanings (e.g., *"How do I change my credentials?"* and *"Where is the password reset page?"*) will produce vectors that are positioned close to each other in vector space, even if they share zero identical words.

4. Setting Up the Local Vector Database (ChromaDB)

Instead of searching through a traditional relational table or text files, we index these embeddings in a specialized **Vector Store** like **ChromaDB**:

- ChromaDB stores your text chunks, their generated mathematical embeddings, and associated metadata (e.g., source file name, page number) together.
- In-Memory vs. Persistent DB: During testing, we can initialize Chroma in memory, but for production systems, we configure a persistent local folder (e.g., `./chroma_db`) to prevent re-indexing the files during every session.

5. Executing Semantic Similarity Search

When a customer submits a support query, the RAG system performs a real-time vector search:

1. **Embed the Query**: The user's query Q is converted into an embedding vector using the exact same `text-embedding-004` model.
2. **Distance Calculation**: The database calculates the similarity score between the query vector Q and all indexed document chunk vectors D using **Cosine Similarity**:$$\text{Similarity}(Q, D) = \frac{Q \cdot D}{\|Q\| \|D\|}$$
3. **K-Nearest Neighbors Retrieval**: The database filters and returns the top-k (typically $k=3$) document chunks that yield the highest similarity scores, along with their metadata. This matched context is then passed to the Persona-Adaptive Generator.

Step 4: Persona-Adaptive Generator

Now, combine the user's input, the classified persona, and the retrieved document chunks inside an custom instructions system prompt.

- **Technical Expert Template**: Instruct the LLM to give granular, systematic details, code parameters, step-by-step logs, and architectural diagnostics.
- **Frustrated User Template**: Instruct the LLM to begin with clear, empathetic validation ("I understand how inconvenient this is..."), use bulleted actions, and avoid long-winded technical jargon.
- **Business Executive Template**: Instruct the LLM to provide a direct answer, a timeline for resolution, operational impact details, and minimal technical explanations.

Step 5: Escalation Check and Human Handoff

If the agent cannot safely solve the query, it must flag the conversation for human intervention.

- **Escalation Triggers**:
    1. *Low Retrieval Confidence*: If the top retrieved document score falls below a threshold (e.g., Cosine Similarity < 0.45).
    2. *Sensitive Topics*: If the classification engine detects billing, refund demands, legal concerns, or account modifications.
    3. *Repeated Frustration*: If sentiment metrics show unresolved frustration over several consecutive turns.

- **Handoff Output**: Generate a highly structured JSON report containing a summary of the customer's issue, attempted troubleshooting configurations, and recommendations for the human responder.

5. Example Code Snippets

These modular snippets demonstrate the clean Python logic required for your backend engine.

A. Persona Classification (Using Structured JSON Output)

This module determines which persona the incoming text represents.

```
import os
import json
from google import genai
from google.genai import types

def classify_customer_persona(user_message: str) -> dict:
    """
    Analyzes the user's message and classifies it into one of the three target personas.
    """
    # Initialize the Gemini GenAI Client
    client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY", ""))

    system_instruction = (
        "You are an advanced classification engine. Your task is to analyze the "
        "sentiment, vocabulary, and tone of an incoming support message and classify "
        "it into exactly one of three customer personas:\n"
        "1. 'Technical Expert': Uses jargon, asks about APIs/code/configs.\n"
        "2. 'Frustrated User': Uses emotional language, exclamation marks, or mentions ur
gency.\n"
        "3. 'Business Executive': Focuses on business impact, ROI, timelines, and brevit
y.\n\n"
        "Provide your evaluation strictly in the requested JSON structure."
    )

    # Define structured schema output
    response_schema = {
        "type": "OBJECT",
        "properties": {
            "persona": {
                "type": "STRING",
                "enum": ["Technical Expert", "Frustrated User", "Business Executive"]
            },
            "confidence": {"type": "NUMBER"},
            "reasoning": {"type": "STRING"}
```

```
        },
        "required": ["persona", "confidence", "reasoning"]
    }

    response = client.models.generate_content(
        model='gemini-2.5-flash-preview-09-2025',
        contents=user_message,
        config=types.GenerateContentConfig(
            system_instruction=system_instruction,
            response_mime_type="application/json",
            response_schema=response_schema,
            temperature=0.1
        )
    )

    return json.loads(response.text)

# Example usage check
if __name__ == "__main__":
    test_msg = "Our production API key stopped working with a 401 Unauthorized block. Che
ck our logs immediately."
    result = classify_customer_persona(test_msg)
    print(json.dumps(result, indent=2))
```

B. Vector Database Ingestion & Retrieval Pipeline

This chunk loads support documentation, indexes it into a vector store using embeddings, and retrieves relevant chunks.

```
import os
from langchain.text_splitter import RecursiveCharacterTextSplitter
from google import genai
import chromadb

class LocalRAGPipeline:
    def __init__(self, db_dir="./chroma_db"):
        self.client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY", ""))
        self.chroma_client = chromadb.PersistentClient(path=db_dir)
        self.collection = self.chroma_client.get_or_create_collection(name="support_kb")

    def get_embedding(self, text: str) -> list:
        """Helper to call Gemini Embedding models."""
        response = self.client.models.embed_content(
            model="text-embedding-004",
            contents=text
        )
        return response.embeddings[0].values
```

```
    def ingest_document(self, doc_name: str, content: str):
        """Split document and add the chunks to the vector database."""
        splitter = RecursiveCharacterTextSplitter(chunk_size=400, chunk_overlap=40)
        chunks = splitter.split_text(content)

        for idx, chunk in enumerate(chunks):
            embedding = self.get_embedding(chunk)
            chunk_id = f"{doc_name}_chunk_{idx}"

            self.collection.add(
                ids=[chunk_id],
                embeddings=[embedding],
                metadatas=[{"source": doc_name, "chunk_index": idx}],
                documents=[chunk]
            )

    def retrieve_context(self, query: str, top_k: int = 2) -> list:
        """Perform search based on cosine similarity scores."""
        query_vector = self.get_embedding(query)

        results = self.collection.query(
            query_embeddings=[query_vector],
            n_results=top_k
        )

        # Format clean lists of context outputs for prompt insertion
        retrieved_items = []
        if results and results['documents']:
            for i in range(len(results['documents'][0])):
                retrieved_items.append({
                    "text": results['documents'][0][i],
                    "source": results['metadatas'][0][i]['source'],
                    # Chroma returns distance metrics, translate to a simulated confidenc
e score
                    "score": 1.0 - (results['distances'][0][i] if results['distances'] el
se 0.0)
                })
        return retrieved_items
```

C. Adaptive Response Generator & Escalation Check

This file orchestrates the retrieved context and persona classification to compile custom instructions and check for escalation constraints.

```
import os
import json
from google import genai
from google.genai import types
```

```
def generate_adaptive_response(user_query: str, persona: str, context_chunks: list) -> di
ct:
    """
    Generates a personalized response matching the classified user archetype.
    If context confidence is too low, the issue is flagged for escalation.
    """
    # 1. Establish the Escalation Check
    confidence_threshold = 0.40
    best_score = max([chunk["score"] for chunk in context_chunks]) if context_chunks else
0.0

    # Trigger escalation criteria if retrieval accuracy is poor
    if best_score < confidence_threshold or len(context_chunks) == 0:
        return {
            "escalated": True,
            "response": "I apologize, but I am unable to locate the precise solution to y
our request. I am connecting you with a live human support specialist.",
            "handoff_summary": generate_handoff_summary(user_query, persona, context_chun
ks)
        }

    # 2. Select System Prompt instruction set depending on classified persona
    if persona == "Technical Expert":
        persona_instructions = (
            "You are a Senior Systems Engineer. Provide clear root-cause analysis, "
            "configuration specifications, and precise API pathways or code blocks. "
            "Keep technical descriptions exact and structured."
        )
    elif persona == "Frustrated User":
        persona_instructions = (
            "You are a deeply empathetic, reassuring Customer Care Specialist. "
            "Begin with a warm, genuine validation of their difficulty. Use straightforwa
rd, "
            "reassuring, and simple action-oriented bullet steps. Avoid confusing jargo
n."
        )
    else:  # Business Executive
        persona_instructions = (
            "You are a concise Client Relations Director. Focus on direct business outcom
es, "
            "impact summaries, and timelines for resolution. Keep responses extremely "
            "brief, professional, and skip unnecessary configuration details."
        )

    # 3. Assemble complete context-grounded system prompt
    context_text = "\n\n".join([f"Source [{c['source']}]: {c['text']}" for c in context_c
hunks])

    full_system_prompt = (
        f"{persona_instructions}\n\n"
        "CRITICAL RULES:\n"
```

```
        "- Base your response ONLY on the provided context.\n"
        "- Do not hallucinate or assume facts not found in the documents.\n\n"
        f"FACTUAL CONTEXT DOCUMENTS:\n{context_text}"
    )

    client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY", ""))

    response = client.models.generate_content(
        model='gemini-2.5-flash-preview-09-2025',
        contents=user_query,
        config=types.GenerateContentConfig(
            system_instruction=full_system_prompt,
            temperature=0.2
        )
    )

    return {
        "escalated": False,
        "response": response.text,
        "handoff_summary": None
    }

def generate_handoff_summary(user_query: str, persona: str, context_chunks: list) -> str:
    """Compiles detailed, structured JSON handoff data for an escalating support ticke
t."""
    handoff_data = {
        "persona": persona,
        "detected_issue": user_query[:100] + "...",
        "retrieved_sources": [c["source"] for c in context_chunks],
        "confidence_score": max([c["score"] for c in context_chunks]) if context_chunks e
lse 0.0,
        "recommended_action": "Review system error codes, check API logs, and contact use
r directly."
    }
    return json.dumps(handoff_data, indent=2)
```

6. Testing Your Implementation

Verification Scenarios

Test your agent with these five standard input scenarios to ensure your logic works correctly:

#	User Message	Expected Persona	Expected Behavior
1	*"Where is the guide to clear cookies? It's been an*	**Frustrated User**	Empathize with their trouble, validate the

#	User Message	Expected Persona	Expected Behavior
	hour and nothing is loading on your interface!"		inconvenience, and list clear, simple troubleshooting steps.
2	*"What are the header parameter requirements for your bearer token auth implementation?"*	**Technical Expert**	Output code blocks, detailed parameters, and raw HTTP header details.
3	*"Our operational uptime is decreasing. We need a timeline of when billing disputes are resolved."*	**Business Executive**	Keep it highly professional, short, and focused on estimated resolution times and business impacts.
4	*"I'm experiencing an issue with your database integration that's causing internal errors."*	**Technical Expert**	Retrieve relevant documentation and outline step-by-step resolution pathways.
5	*"My billing statement has unexpected duplicate charges. I demand an immediate refund!"*	**Frustrated User**	**Trigger Escalation**: Detect billing sensitivity and generate a structured human handoff JSON.

Performance Optimization Tips

1. **Exponential Backoff**: When building your interactive UI loop, wrap LLM calls in a retry handler to manage API rate limits:

```
import time
import random

def call_gemini_with_backoff(func, *args, max_retries=5, **kwargs):
    for attempt in range(max_retries):
        try:
            return func(*args, **kwargs)
        except Exception as e:
            if attempt == max_retries - 1:
                raise e
            sleep_time = (2 ** attempt) + random.uniform(0, 1)
            time.sleep(sleep_time)
```

2. **Keep Embeddings Local**: For production workloads, generate and save the document index locally first. Do not regenerate the embedding index inside every chat turn. Only load and query the persistent database at runtime.

Submission Guidelines

You must submit **all three** of the following:

1. **GitHub Repository Link** — A public repo containing the complete project source code and the analysis document in the `README.md`.
2. **Published / Deployed Project Link** — A live, working URL
3. **Screen Recording Link** — A short video (3–5 minutes) explaining your project, its features, and a brief walkthrough of the code. Upload to Google Drive, Loom, or YouTube (unlisted is fine).